

International Conference on Computational Science, ICCS 2012

# A Novel, Evolutionary, Simulated Annealing inspired Algorithm for the Multi-Objective Optimization of Combinatorial Problems

Elias D. Nino<sup>a,b</sup>, Carlos J. Ardila<sup>b</sup>, Anangelica Chinchilla<sup>c</sup><sup>a</sup>*enino@vt.edu, Department of Computer Science, Virginia Tech, Blacksburg, United States of America*<sup>b</sup>*{enino,cardila}@uninorte.edu.co, Department of Computer Science, Universidad del Norte, Barranquilla, Colombia*<sup>c</sup>*Department of Industrial Engineering, Universidad del Norte, Barranquilla, Colombia*

---

## Abstract

This paper states a novel hybrid-metaheuristic based on deterministic swapping, evolutionary algorithms and simulated annealing inspired algorithms for the multi-objective optimization of combinatorial problems. The proposed algorithm is named EMSA. It is an improvement of MODS algorithm. Unlike MODS, EMSA works using a search direction given by the assignation of weights to each objective function of the combinatorial problem to optimize. Lastly, EMSA is tested using well know instances of the Bi-Objective Traveling Salesman Problem (BTSP) from TSPLIB. Its results were compared with MODS metaheuristic (its predecessor). The comparison was made using metrics from the specialized literature such as Spacing, Generational Distance, Inverse Generational Distance and Non-Dominated Generation Vectors. In every case, the EMSA results on the metrics were always better and in some of those cases, the superiority was 100%.

**Keywords:** Combinatorial Optimization, Genetic Algorithms, Simulated Annealing, Multi-objective Optimization  
**2008 MSC:** 68R05, 68R10, 90-08, 90C27, 90C29

---

## 1. Introduction

Combinatorial optimization is a branch of optimization. Its domain is optimization problems where the set of feasible solutions is discrete or can be reduced to a discrete one, and the goal is to find the best possible solution. One of the most classical problems in the Combinatorial Optimization Field is the Traveling Salesman Problem (TSP), it has been analyzed for years, either in a Mono or Multi-objective way. Formally, TSP is defined as follows:

$$\min \sum_{i=1}^n \sum_{j=1}^n C_{ij} \cdot X_{ij}, \quad (1)$$

subject to:

$$\sum_{j=1}^n X_{ij} = 1, \forall i = 1, \dots, n, \quad (2a)$$

$$\sum_{i=1}^n X_{ij} = 1, \forall j = 1, \dots, n, \quad (2b)$$

$$\sum_{i \in \kappa} \sum_{j \in \kappa} X_{ij} \leq |\kappa| - 1, \forall \kappa \subset \{1, \dots, n\}, \quad (2c)$$

$$X_{ij} = 0, \forall i, j, \quad (2d)$$

where  $C_{ij}$  is the cost of the path  $X_{ij}$  and  $\kappa$  is any nonempty proper subset of the cities  $1, \dots, m$ . Equation (1) represents the objective function. The goal is the optimization of the overall cost of the tour. Equations (2a), (2b) and (2d) fulfills the constrain of visiting each city only once. Lastly, equation (2c) establishes the subsets of solutions, avoiding cycles in the tour.

TSP has an important impact on different sciences and fields. For instance in Operations Research and Theoretical Computer Science, problems such as Heterogeneous Machine Scheduling[1] and The Hard Scheduling Optimization[2] had been derived from TSP. Although several algorithms have been implemented to solve TSP, there is no one that optimal solves it.

This paper is structured as follows: Section 2 shows important definitions to understand the multi-objective combinatorial optimization field and the metaheuristic approximation. Section 3 and 4 discuss an hybrid evolutionary metaheuristic to optimize multi-objective combinatorial problems. Finally, Section 5 and 6 discuss the Experimental Results of the proposed Algorithm using Multi-objective Metrics from the specialized literature.

## 2. Preliminaries

### 2.1. Multiobjective Optimization

The Multi-Objective Optimization (MO) consists in two or more objectives functions to optimize and a set of constraints. Mathematically, the MO model is defined as follows:

$$\text{optimize } F(X) = \{f_1(X), f_2(X), \dots, f_n(X)\}, \quad (3)$$

subject to:

$$H(X) = 0, \quad (4a)$$

$$G(X) \leq 0, \quad (4b)$$

$$X_l \leq X \leq X_u, \quad (4c)$$

where  $F(X)$  is the set of objective functions,  $H(X)$  and  $G(X)$  are the constraints of the problem. Lastly,  $X_l$  and  $X_u$  are the bounds for the set of variables  $X$ .

Unlike to Mono-Objective Optimization, Multi-objective Optimization deals with searching a set of optimal solutions instead of an optimal solution.

### 2.2. Genetic Algorithms

Genetic Algorithms are Algorithms based on the Theory of Natural Selection. Thus, Genetic Algorithms mimics the real behavior of real evolutionary systems through three basic steps: Given a set of Initial Solutions  $S$

*Step 1. Selection.* Select solutions from a population. In pairs, select two solutions  $x, y \in S$

*Step 2. Crossover.* Cross the selected solutions avoiding local optimums.

*Step 3. Mutation.* Perturbs the new solutions found for increasing the population. The perturbation can be done according to the representation of the solution. In this step, good solutions are added to  $S$

The most known Genetic Algorithms from the literature are the Non-Dominated Sorting Genetic Algorithm[3] (NSGA-II) and the Strength Pareto Evolutionary Algorithm 2[4] (SPEA 2).

### 2.3. Simulated Annealing inspired Algorithms

Simulated Annealing[5] is a generic probabilistic metaheuristic based in the Annealing in Metallurgy. It explores the neighborhood of solutions being flexible with ill solutions. That is mean, accepting bad solutions as well as good solution, but only in the first iterations, when the temperature is high. The acceptance of a bad solution is based on the Boltzmann Probabilistic Distribution:

$$P(x) = e^{\left(-\left(\frac{E}{T_i}\right)\right)} \quad (5)$$

Where  $E$  is the change of the Energy and  $T_i$  is the temperature in the moment  $i$ . In the first level of the temperature, bad solutions are accepted as well as good solutions. However, when the temperature falls, Simulated Annealing behaves similar to Tabu Search[6].

### 2.4. Metaheuristic of Deterministic Swapping

Metaheuristic Of Deterministic Swapping (MODS) [7] is a local search strategy that explores the Feasible Solution Space of a Combinatorial Problem supported in a data structure named Multi Objective Deterministic Finite Automata (MDFA) [8]. A MDFA is a Deterministic Finite Automata that allows the representation of the feasible solution space of a Combinatorial Problem. Formally, a MDFA is defined as follows:

$$M = (Q, \Sigma, \delta, Q_0, F(X)) \quad (6)$$

Where  $Q$  represents all the set of states of the automata (feasible solution space),  $\Sigma$  is the input alphabet that is used for  $\delta$  (transition function) to explore the feasible solution space of a combinatorial problem,  $Q_0$  contains the initial set of states (initial solutions) and  $F(X)$  are the objectives to optimize.

We re-defined the meta-heuristic with multi-optimization common variables. Thus, the template algorithm of MODS is defined as follows:

*Step 1.* Create the initial set of solutions  $S_0$  using a heuristic relative to the problem to solve.

*Step 2.* Set  $S$  as  $S_0$  and  $S^*$  as  $\phi$ .

*Step 3.* Select a random solution  $s \in S$  or  $s \in S^*$

*Step 4.* Perturb  $s$  in order to know its neighborhood. For those solutions that are non-dominated by elements of  $S$ , join to this set. In addition, add to  $S^*$  those solutions found that dominated at least one element from  $S$ .

*Step 5.* Check stop condition, go to 3.

$S$  is the solution set of non-dominated solutions and  $S^*$  is the set of Elitism Solutions.

## 3. An hybrid metaheuristic for the Multi-Objective optimization of Combinatorial Problems

Evolutionary Metaheuristic of Simulated Annealing (EMSA) is an improvement of MODS metaheuristic. EMSA is defined as a hybrid metaheuristic based on Genetic Algorithms, Simulated Annealing and Deterministic Swapping[9] for the multi-objective optimization of combinatorial problems. Its main propose consists in optimizing a combinatorial problem using a Search Direction and an Angle Improvement. EMSA is based in the next Automata:

$$M_{EMSA} = (S, S_0, P(s), A(n), C(q, r, k), F(X)) \quad (7)$$

Alike MODS,  $S_0$  is the set of initial solutions,  $S$  is the feasible solution space (unknown) and  $F(X)$  are the functions of the combinatorial problem. In addition,  $P(s)$  is the *permutation function* ( $P(s) : S \rightarrow S$ ),  $C(q, r, k)$  is the crossing function ( $C(q, r, k) : S \rightarrow S$ , having  $q, r \in S$  and  $k \in \mathbb{N}$ ) and  $A(n)$  is the weighted function ( $A(n) : \mathbb{N} \rightarrow \mathbb{R}^n$ , having  $n \in \mathbb{N}$ )

By definition, function  $A$  receives a natural number as parameter and it returns a vector with the weights. The weight values are randomly generated (with an uniform distribution) Every value represents the weight to assign to each objective function. The weight values fulfill the next constrain:

$$\sum_{i=1}^n \alpha_i = 1, 0 \leq \alpha_i \leq 1, \quad (8)$$

where  $\alpha_i$  is the weight assigned to function  $i$ . As can be seen, the weights implicitly allow to establish the search direction in order to optimize  $F(X)$ . Hence, when the weights values are changed, the angle of optimization is changed and a new search direction is obtained. Due to this, (3) can be rewritten as follows:

$$F(X) = \sum_{i=1}^n \alpha_i \cdot f_i(X), \quad (9)$$

where  $n$  is the number of objectives of the problem. The weights fulfills the constrain established in (8).

Lastly, EMSA template is defined as follows:

*Step 1.* Creating sets. Set  $S_0$  as the set of Initial Solutions. Set  $S_\phi$  and  $S_*$  as  $S_0$ .

*Step 2.* Settings parameters. Set  $T$  as the initial temperature,  $n$  as the number of objectives of the problem and  $\rho$  as the cooler factor.

*Step 3.* Decreasing the temperature. If  $T$  is equal to 0 then got to 9, else set  $T_{i+1} = \rho \times T_i$ , and go to step 4.

*Step 4.* Selection and Direction. Randomly select  $s \in S_\phi \vee s \in S_*$ , set  $W = A(n) = \{w_1, w_2, \dots, w_n\}$  and go to step 5.

*Step 5.* Mutation. Set  $s' = P(s)$ , add to  $S_\phi$  and  $S_*$  according to the next rules:

$$S_\phi = S_\phi \cup \{s'\} \Leftrightarrow (\nexists r \in S_\phi)(r \text{ is better than } s'), \quad (10)$$

$$S_* = S_* \cup \{s'\} \Leftrightarrow (\exists r \in S_*)(s' \text{ is better than } r), \quad (11)$$

if  $S_\phi$  has at least one element that dominated to  $s'$  go to step 5, otherwise go to step 7.

*Step 6.* Trying Dominated Solutions. Randomly generate a number  $n \in [0, 1]$ . Set  $z$  as follows:

$$z = e^{-(\gamma/T_i)}, \quad (12)$$

where  $T_i$  is the temperature value in moment  $i$  and  $\gamma$  is defined as follows:

$$\gamma = \sum_{i=1}^n w_i \cdot f_i(s_X) - \sum_{i=1}^n w_i \cdot f_i(s'_X), \quad (13)$$

where  $s_X$  is the vector  $X$  of solution  $s$ ,  $s'_X$  is the vector  $X$  of solution  $s'$ ,  $w_i$  is the weight assigned to the function  $i$  and  $n$  is the number of objectives of the problem. If  $n < z$  then set  $s$  as  $s'$  and go to step 4 else go to step 7.

*Step 7.* Crossover. Set  $s' = C(s, s', k)$ , where  $1 \leq k \leq \|s_X\|$ , then go to step 5.

*Step 8.* Removing Dominated Solutions. Remove the dominated solutions of each set ( $S_*$  and  $S_\phi$ ), then go to step 3.

*Step 9.* Finishing.  $S_\phi$  has the non-dominated solutions.

#### 4. Computational Cost of the Metaheuristic Proposed

For the computation of EMSA complexity, we consider assignments, arithmetic operations and other simple instructions as  $O(k)$ , where  $k$  is a constant. Due to this, the external cycle (step 3), the creation of a search direction (step 4), the crossover (step 7), looking for dominated solutions (step 5) and remove the dominated solution of a set (step 8), all of them have a value of  $O(n)$  where  $n$  is a large number. Thus, the equation that represents the EMSA space equation can be written as follows:

$$T_{EMSA}(n) = \sum_{i=1}^n (4 \cdot n + O(k)), \quad (14)$$

therefore,

$$T_{EMSA}(n) = 4 \cdot n^2 + n \cdot O(k), \quad (15)$$

applying the well-known Big-O notation to equation (15) we find an upper bound for EMSA:

$$T_{EMSA}(n) = O(n^2). \quad (16)$$

On the other hand, the lower bound is calculated using (15). Hence:

$$\begin{aligned} T_{EMSA}(n) &= 4 \cdot n^2 + n \cdot O(k), \\ &\geq 4 \cdot n + n \cdot O(k), \\ &= (4 + O(k)) \cdot n = (c) \cdot n, \end{aligned} \quad (17)$$

therefore, applying the well-known  $\Omega$  notation to equation (17) we find a lower bound for EMSA:

$$T_{EMSA}(n) = \Omega(c \cdot n) \quad (18)$$

## 5. Experimental Results

### 5.1. Performance Metrics

There are metrics for measuring the quality of a set of optimal solutions and the performance of an algorithm. Most of them use two Pareto fronts. The first one is  $PF_{true}$  and it refers to the real optimal solutions of a combinatorial problem. The second is  $PF_{know}$  and it represents the optimal solutions found by an algorithm (true solutions).

**Generation of Non-dominated Vectors (GNDV)** It measures the number of non-dominated solutions generated by an algorithm.

$$GNDV = \|PF_{know}\|$$

**Rate of Generation of No-dominated Vectors (RGNDV)** This metric measures the proportion of the non-dominated solutions (equation (19)) generated by an algorithm and the true solutions.

$$RGNDV = \left( \frac{GNDV}{\|PF_{true}\|} \right) \cdot 100\%$$

**Real Generation of Non-dominated Vectors (ReGNDV)** This metric measures the number of true solutions found by an algorithm.

$$ReGNDV = \|\{y | y \in PF_{know} \wedge y \in PF_{true}\}\|$$

**Generational Distance (GD) and Inverse Generational Distance (IGD)** These metrics measure the distance between  $PF_{know}$  and  $PF_{true}$ . They allow to compute the error rate of an algorithm in terms of the distance to the true solutions.

$$\begin{aligned} GD(PF_{know}, PF_{true}) &= \left( \frac{1}{\|PF_{know}\|} \right) \cdot \left( \sum_{i=1}^{\|PF_{know}\|} d_i \right)^{(1/p)}, \\ IGD(PF_{know}, PF_{true}) &= \left( \frac{1}{\|PF_{true}\|} \right) \cdot \left( \sum_{i=1}^{\|PF_{know}\|} d_i \right), \end{aligned}$$

where  $d_i$  is the smallest euclidean distance between the solution  $i$  of  $PF_{know}$  and the solutions of  $PF_{true}$ .  $p$  is the dimension of the combinatorial problem. **Spacing (S)** It measures the range variance of neighboring solutions in  $PF_{know}$

$$S(PF_{know}) = \left( \frac{1}{\|PF_{know}\| - 1} \right)^2 \cdot \left( \sum_{i=1}^{\|PF_{know}\|} (\bar{d} - d_i)^2 \right)^{(1/p)}, \quad (19)$$

where  $d_i$  is the smallest euclidean distance between the solution  $s_i \in PF_{know}$  to  $s_j \in PF_{know}$  having  $s_i \neq s_j$ ,  $\bar{d} = \frac{1}{\|PF_{know}\|} \sum_{i=1}^{\|PF_{know}\|} d_i$  and  $p$  is the dimension of the combinatorial problem.

**Error Rate ( $\varepsilon$ )** It estimates the error rate regarding the precision of the true solutions as follows:

$$\varepsilon = \left( \frac{\|PF_{true}\|}{ReGNDV} \right) \cdot 100\% \quad (20)$$

## 5.2. Experimental Settings

The proposed algorithm was tested using wellknown instances from the Bi-objective Traveling Salesman Problem (BTSP) taken from TSPLIB [10]. The input parameters set to the algorithms in comparison are shown in table 1. The test of the algorithms was made using a Dual Core Computer with 2 Gb RAM. Each algorithm was running ten times and the best non-dominated solutions were taken for making a real comparison. The optimal solutions were constructed based in the best non-dominated solutions of MODS and EMSA for each instance worked.

Table 1: Parameters setting for each compared algorithm.

| Algorithm | Iterations | Perturbations | Initial Temperature | Cooler Value |
|-----------|------------|---------------|---------------------|--------------|
| MODS      | 100        | 80            | NA                  | NA           |
| EMSA      | 100        | NA            | 1000                | 0.95         |

The test made with bi-objectives instances is shown in table 2. In addition, a graphical comparison is shown in figure 1.

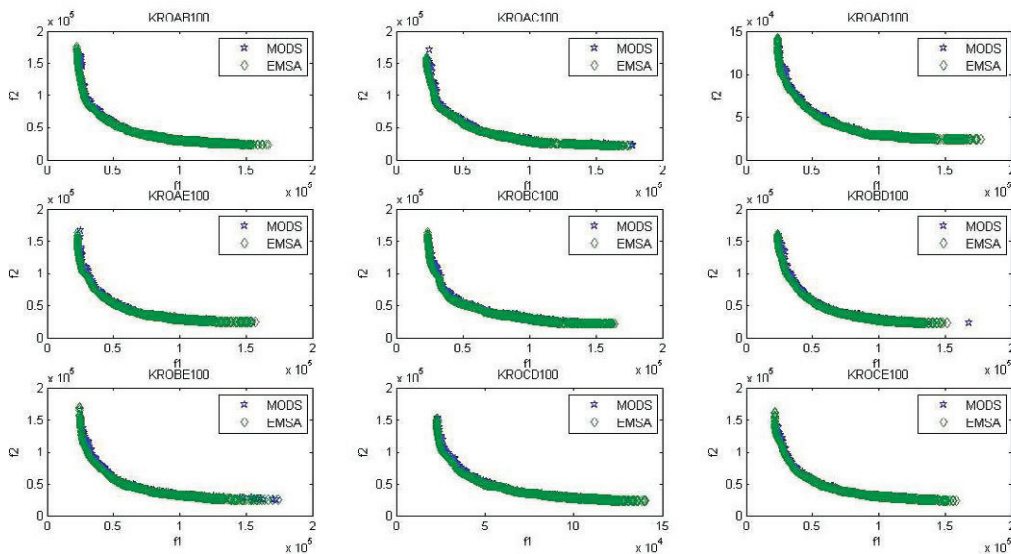


Figure 1: Graphical comparison between MODS and EMSA Pareto fronts for the BTSP.

Table 2: Performance of the algorithms for the BTSP using MultiObjective Optimization Metrics.

| INSTANCE | ALGORITHM | GNDV | RGNDV | ReGNDV | $\left(\frac{ReGNDV}{GNDV}\right)\%$ | S     | GD     | IGD      | $\varepsilon$ |
|----------|-----------|------|-------|--------|--------------------------------------|-------|--------|----------|---------------|
| KROAB100 | MODS      | 289  | 0.034 | 0      | 0%                                   | 0.019 | 14.945 | 2200.006 | 100%          |
|          | EMSA      | 8479 | 1     | 8479   | 100%                                 | 0.001 | 0.019  | 3.175    | 0%            |
| KROAC100 | MODS      | 217  | 0.031 | 0      | 0%                                   | 0.034 | 19.12  | 2451.132 | 100%          |
|          | EMSA      | 7023 | 1     | 7023   | 100%                                 | 0.001 | 0.028  | 5.484    | 0%            |
| KROAD100 | MODS      | 281  | 0.045 | 0      | 0%                                   | 0.014 | 11.997 | 1807.158 | 100%          |
|          | EMSA      | 6289 | 1     | 6289   | 100%                                 | 0.002 | 0.032  | 6.426    | 0%            |
| KROAE100 | MODS      | 283  | 0.044 | 0      | 0%                                   | 0.053 | 13.251 | 2183.784 | 100%          |
|          | EMSA      | 6440 | 1     | 6440   | 100%                                 | 0.001 | 0.028  | 5.019    | 0%            |
| KROBC100 | MODS      | 298  | 0.043 | 0      | 0%                                   | 0.016 | 13.777 | 2436.033 | 100%          |
|          | EMSA      | 6919 | 1     | 6919   | 100%                                 | 0.001 | 0.034  | 7.992    | 0%            |
| KROBD100 | MODS      | 241  | 0.041 | 0      | 0%                                   | 0.024 | 14.607 | 2088.49  | 100%          |
|          | EMSA      | 5934 | 1     | 5934   | 100%                                 | 0.001 | 0.037  | 8.16     | 0%            |
| KROBE100 | MODS      | 280  | 0.048 | 0      | 0%                                   | 0.031 | 13.395 | 2424.672 | 100%          |
|          | EMSA      | 5802 | 1     | 5802   | 100%                                 | 0.002 | 0.037  | 7.848    | 0%            |
| KROCD100 | MODS      | 286  | 0.045 | 0      | 0%                                   | 0.018 | 11.887 | 1834.388 | 100%          |
|          | EMSA      | 6301 | 1     | 6301   | 100%                                 | 0.001 | 0.029  | 5.229    | 0%            |
| KROCE100 | MODS      | 224  | 0.049 | 1      | 0.4%                                 | 0.028 | 12.639 | 1737.247 | 99%           |
|          | EMSA      | 4613 | 1     | 4613   | 100%                                 | 0.003 | 0.002  | 0.014    | 0%            |
| KRODE100 | MODS      | 228  | 0.029 | 0      | 0%                                   | 0.048 | 16.01  | 1720.449 | 100%          |
|          | EMSA      | 7745 | 1     | 7745   | 100%                                 | 0.001 | 0.026  | 5.227    | 0%            |

## 6. Conclusions

EMSA is an improvement of MODS metaheuristic, it uses different search directions in order to avoid dominated solutions. EMSA is useful for solving problems such as TSP and its applications. Moreover, the proposed algorithm shows a better computational performance than MODS in BTSP problems with the same computational cost. Furthermore, EMSA uses a search direction given by a linear combination of the functions to optimize. With regard to local optimums, they are avoided (tried) in two manners, the first one is using guessing with bad solutions and the last is using crossover between solutions (non-dominated as well as dominated solutions). Lastly, metrics from the specialized literature were used for the comparison of EMSA and MODS solutions. The results shows that EMSA has a better computational performance than MODS and in some cases the superiority was 100% out of 100%.

## References

- [1] G. H. Kim, C. Lee, Genetic reinforcement learning approach to the heterogeneous machine scheduling problem, Robotics and Automation, IEEE Transactions on 14 (6) (1998) 879–893. doi:10.1109/70.736772.
- [2] E. D. Niño, C. Ardila, A. Perez, Y. Donoso, A genetic algorithm for multiobjective hard scheduling optimization, International Journal of Computers Communications & Control 5 (5) (2010) 825–836.
- [3] K. Deb, A. Pratap, S. Agarwal, T. Meyarivan, A fast and elitist multiobjective genetic algorithm: Nsga-ii, Evolutionary Computation, IEEE Transactions on 6 (2) (2002) 182–197. doi:10.1109/4235.996017.
- [4] E. Zitzler, M. Laumanns, L. Thiele, Spea2: Improving the strength pareto evolutionary algorithm, Tech. Rep. 103, Computer Engineering and Networks Laboratory (TIK), Swiss Federal Institute of Technology (ETH) Zurich, Gloriastrasse 35, CH-8092 Zurich, Switzerland (May 2001).
- [5] S. Kirkpatrick, C. D. Gelatt, M. P. Vecchi, Optimization by Simulated Annealing, Science, Number 4598, 13 May 1983 220, 4598 (1983) 671–680.  
URL <http://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.18.4175>
- [6] F. Glover, M. Laguna, Tabu Search, Kluwer Academic Publishers, Norwell, MA, USA, 1997.
- [7] E. D. e. a. Niño, Mods: A novel metaheuristic of deterministic swapping for the multi objective optimization of combinatorials problems, Computer Technology and Application 2 (4) (2011) 280–292.
- [8] E. D. Niño, C. Ardila, Y. Donoso, D. Jabba, A novel algorithm based on deterministic finite automaton for solving the mono-objective symmetric traveling salesman problem, International Journal of Artificial Intelligence 5 (A10) (2010) 101–108.
- [9] E. D. Niño, Samods and sagamods: Novel algorithms based on the automata theory for the multi-objective optimization of combinatorial problems, International Journal of Artificial Intelligence - Special issue of IJAI on Metaheuristics in Artificial Intelligence Pending (2012) Pending.
- [10] U. of Heidelberg, Tsplib - office research group discrete optimization - university of heidelberg, <http://comopt.ifi.uni-heidelberg.de/software/TSPLIB95/>.